University of Science - VNUHCM

Faculty of Information Technology

Software Testing

HW3 - Data Generation & Scenario Testing

Software Testing Project Report

Authors:
Lưu Thanh Thuý
(22127410)
ltthuy22@clc.fitus.edu.vn

Supervisors:
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

June 24, 2025

Table of Contents

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Features	Status
Cao Uyển Nhi	22127310	- SignUp	Done
		- Checkout	Done
Lưu Thanh Thuý	22127410	- SignIn	Done
		- User Management	Done
Nguyễn Phước Minh Trí	22127424	- Catalog	Done
		- Categories	Done
Võ Lê Việt Tú	22127435	- MyProfile	Done
		- Order Management	Done
Trần Thị Cát Tường	22127444	- Contact	Done
		- Category Management	Done

2 Scenario Definition

This document outlines the detailed specifications for two key features of the system: **User Sign In** and **User Management**. Each feature includes a comprehensive description, detailed basic and alternate flows, and an expanded scenario table with precise conditions, actions, and expected outcomes. The goal is to provide clear, actionable requirements for developers, testers, and stakeholders.

2.1 Feature 1: User Sign In

2.1.1 Description

The **User Sign In** feature enables registered users to authenticate and access the system securely. The feature is accessible via a dedicated Sign In page, which presents a form requiring a valid email address and password. Upon successful submission, the system verifies the credentials against the database and grants access to a role-based dashboard (either **User** or **Admin**). The feature includes real-time input validation, error handling for invalid inputs, and appropriate feedback to ensure a seamless user experience. Security measures, such as password hashing and rate-limiting for brute-force prevention, are assumed to be implemented at the backend.

2.1.2 Basic Flow

1. **Navigation to Sign In Page**

- The user accesses the Sign In page via a URL (e.g., `/signin`) or a "Sign In" link from the homepage or navigation bar.
- The page loads a form with fields for **Email** and **Password**, a **Sign In** button, and links for "Forgot Password" and "Sign Up."

2. **Input Credentials**

- The user enters their registered email address in the Email field (e.g., `user@example.com`).
- The user enters their password in the Password field (e.g., `P@ssw0rd123`).
- Both fields are marked as required, and the Password field obscures input for security.

3. **Submit Form**

- The user clicks the **Sign In** button to submit the form.
- The frontend performs client-side validation to ensure fields are not empty and the email format is valid (e.g., contains `@` and `.`).

4. **Credential Verification**

- The system sends a secure HTTPS POST request to the backend API (e.g., `/api/auth/signin`) with the email and password.
- The backend queries the database to check if the email exists and verifies the password using a secure hashing algorithm (e.g., bcrypt).

5. **Successful Authentication**:

- If credentials are valid, the backend generates a session token or JSON Web Token (JWT) and returns it to the frontend.
- The system logs the user in and redirects them to their role-based dashboard:
  - **User Role**: Redirects to `/dashboard/user`.
  - **Admin Role**: Redirects to `/dashboard/admin`.

2.1.3 Alternate Flows

2.a. Empty Email or Password Field:

If the user submits the form with either the Email or Password field empty:

- Client-side validation triggers immediately.
- An inline error message appears below the respective field(s):
  - For Email: *"Email is required."*
  - For Password: *"Password is required."*
- The form submission is blocked until both fields are filled.

2.b. Invalid Email Format:

If the user enters an invalid email (e.g., `invalid-email` or `user@.com`):

- Client-side validation detects the incorrect format using a regular expression (e.g., `^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$`).
- An inline error message appears below the Email field: *"Please enter a valid email address."*
- The form submission is blocked until a valid email is provided.

4.a. Unregistered Email:

If the email does not exist in the database:

- The backend returns a 401 Unauthorized response with the message: *"No account associated with this email."*

- The frontend displays this error message below the form.
- The user remains on the Sign In page and can retry or navigate to the Sign Up page.

4.b. Incorrect Password:

If the email exists but the password does not match the stored hash:

- The backend returns a 401 Unauthorized response with the message: *"Incorrect password."*
- The frontend displays this error message below the form.
- The user remains on the Sign In page and can retry or use the "Forgot Password" link.

5.a. System Error:

If a server-side error occurs (e.g., database connectivity issue or timeout):

- The backend returns a 500 Internal Server Error response with the message: *"Login failed due to a system error. Please try again later."*
- The frontend displays this error message below the form.
- The user remains on the Sign In page and can retry after a short delay.
- The system logs the error for debugging purposes.

2.1.4 Scenario Table

ID	Scenario Name	Precondition	Action	Expected Outcome
S1-01	Successful login	User has a registered account with valid credentials.	Enter valid email and password, click Sign In.	User is logged in, redirected to role-based dashboard.
S1-02	Empty email	Email field is empty.	Leave Email blank, enter password, click Sign In.	Inline error below Email field: *"Email is required."* Form submission blocked.
S1-03	Empty password	Password field is empty.	Enter valid email, leave Password blank, click Sign In.	Inline error below Password field: *"Password is required."* Form submission blocked.
S1-04	Invalid email format	Email format is invalid.	Enter invalid email, click Sign In.	Inline error below Email field: *"Please enter a valid email address."* Form submission blocked.
S1-05	Unregistered email	Email is not registered in the system.	Enter unregistered email click Sign In.	Error below form: *"No account associated with this email."* User remains on Sign In page.

ID	Scenario Name	Precondition	Action	Expected Outcome
S1-06	Wrong password	Email exists, but password is incorrect.	Enter valid email, incorrect password, click Sign In.	Error below form: *"Incorrect password."* User remains on Sign In page.
S1-07	System error during login	Backend experiences a failure (e.g., DB down).	Enter valid credentials, click Sign In during simulated backend failure.	Error below form: *"Login failed due to a system error. Please try again later."* User remains on Sign In page.

2.2 Feature 2: User Management (Admin Only)

2.2.1 Description

The **User Management** feature enables administrators to manage user accounts within the system. Accessible only to users with **Admin** privileges, this module allows admins to view a list of all users, add new user accounts, edit existing user details, delete user accounts, and reset user passwords. The module is accessed via a dedicated User Management section in the Admin dashboard, featuring a user-friendly interface with a table view for users and forms for adding/editing accounts. All operations include input validation, role-based access control, and appropriate success/error feedback. The system ensures data integrity by enforcing unique email constraints and preventing unauthorized actions (e.g., self-deletion by admins).

2.2.2 Basic Flow

1. **Access User Management Module**

- An admin logs into the system using valid credentials and navigates to the Admin dashboard (`/dashboard/admin`).
- The admin clicks on the "User Management" link in the sidebar or menu, loading the User Management module (`/admin/users`).
- The module displays a paginated table listing all users with columns for **Name**, **Email**, **Role** (User/Admin), **Status** (Active/Inactive), and **Actions** (Edit, Delete, Reset Password).

2. **Perform User Management Operations**

- **View Users**:
  - The table loads data via an HTTPS GET request to the backend API (e.g., `/api/users`).
  - The admin can sort the table by columns (e.g., Name, Email) or filter by Role or Status.
- **Add New User**:
  - The admin clicks an "Add User" button, opening a modal or new page with a form.
  - The form includes required fields: **Name** (text), **Email** (email), **Password** (text, obscured), **Role** (dropdown: User/Admin), and optional

Status (checkbox: Active/Inactive).
  - The admin fills out the form (e.g., Name: `John Doe`, Email: `john.doe@example.com`, Password: `SecurePass123`, Role: User, Status: Active) and clicks "Save."
  - The system sends an HTTPS POST request to `/api/users` with the form data.
- **Edit Existing User**:
  - The admin clicks the "Edit" button next to a user in the table, opening a pre-filled form with the user's details.
  - The admin updates fields (e.g., changes Role from User to Admin or updates Name) and clicks "Save."
  - The system sends an HTTPS PATCH request to `/api/users/{userId}` with the updated data.
- **Delete User**:
  - The admin clicks the "Delete" button next to a user in the table.
  - A confirmation dialog appears: *"Are you sure you want to delete [User Name]?"*
  - The admin confirms the deletion.
  - The system sends an HTTPS DELETE request to `/api/users/{userId}`.
- **Input Validation and Database Update**:
  - The frontend validates all inputs client-side (e.g., required fields, email format, password strength).
  - The backend validates all inputs server-side (e.g., unique email, role permissions).
  - Upon successful validation, the database updates and the system returns a 200 OK response with a success message (e.g., "User created successfully.").
- **Feedback Confirmation**:
  - Success messages are displayed (e.g., "User [Action] added successfully!") for each operation after a successful operation.
  - The table refreshes automatically to reflect the changes.
  - Error messages are displayed for failed operations (see Alternate Flows).

2.2.3 Alternate Flows

1.a. Admin Non- Access Denied:

If a non-admin user (e.g., Role: User) attempts to access the User Management module (`/admin/users`):

- The system checks the user's role via the backend token or session token.
- The backend returns a 403 Forbidden response with the message: *"Access denied. Admin privileges required."*
- The frontend redirects the user to an error page (e.g., `/access-denied`) with the message: *"You do not have permission to access this section."*

2.a. Add User with Missing Required Fields:

If the admin submits the Add/Edit or Edit User form with missing required fields (e.g., Name or Email blank):

- Client-side validation triggers immediately.
- Inline error messages appear below each missing field:
  - For Name: *"Name is required."*
  - For Email: *"Email is required."*
  - For Password (on Add): *"Password is required."*
- The form submission is blocked until all required fields are filled.

2.b. Add User with Existing Email:

If the admin attempts to add or edit a user with an email already in the database (e.g., `john.doe@example.com`):

- The backend validates the email uniqueness and returns a 409 Conflict response with the message: *"Email already in use."*
- The frontend displays this error message below the Email field.
- The form remains open for the admin to correct the email.

2.c. Delete Own Admin Account:

If the admin attempts to delete their own account:

- The backend checks the user ID against the authenticated admin's ID and returns a 403 Forbidden response with the message: *"Operation not permitted. You cannot delete your own account."*
- The frontend displays this error message in the confirmation dialog, and the deletion is blocked.

3.a. System Error During CRUD Operation:

If a server-side error occurs during any CRUD operation (e.g., database failure, network timeout):

- The backend returns a 500 Internal Server Error response with the message: *"Operation failed. Please retry later."*
- The frontend displays this error message as a toast notification or below the form.
- The admin remains on the User Management page and can retry the operation.
- The system logs the error for debugging purposes.

2.2.4 Scenario Table

ID	Scenario Name	Precondition	Action	Expected Outcome
S2-01	View all users	Admin is logged in and on User Management page.	Access `/admin/users`.	Paginated table displays users with columns: Name, Email, Role, Status, Actions. Sorting and filtering available.
S2-02	Add valid user	Admin is on Add User form.	Enter valid data, click Save.	Success message: *"User added successfully."* Table updates with new user.

ID	Scenario Name	Precondition	Action	Expected Outcome
S2-03	Add user with existing email	Email already exists in DB.	Enter existing email, click Save.	Error below Email field: *"Email already in use."* Form remains open.
S2-04	Edit user details	Admin is on Edit User form for existing user.	Update fields, click Save.	Success message: *"User updated successfully."* Table reflects changes.
S2-05	Delete user	Admin is on User Management page.	Click Delete next to a user, confirm in dialog.	Success message: *"User deleted successfully."* User removed from table.
S2-06	Add user missing required fields	Admin is on Add User form.	Leave Name or Email blank, click Save.	Inline errors: *"Name is required."* or *"Email is required."* Form submission blocked.
S2-07	Non-admin access	Non-admin user is logged in.	Attempt to access `/admin/users`.	Redirect to error page with message: *"Access denied. Admin privileges required."*
S2-08	Delete own admin account	Admin is on User Management page.	Click Delete next to own account, confirm in dialog.	Error in dialog: *"Operation not permitted. You cannot delete your own account."*
S2-09	Backend failure on user operation	Backend experiences failure (e.g., DB down).	Attempt any CRUD operation during failure.	Error notification: *"Operation failed. Please retry later."* Admin remains on page.

3 Use of AI Tools

3.1 Tool Information

Tool Name: GitHub Copilot

3.2 Prompts Used

The following prompts were utilized during the development of this scenario document:

1. **Initial Structure Generation:**

- *"Create a detailed scenario document for User Sign In and User Management features with comprehensive flow descriptions and test cases"*
- *"Generate alternate flows for authentication failures and validation errors in user management systems"*

2. **Test Case Refinement:**

- *"Create specific test scenarios for admin role validation and access control in user management"*
- *"Generate edge cases for form validation including empty fields, invalid formats, and duplicate entries"*

3. **Documentation Enhancement:**

- *"Improve the clarity and technical detail of authentication flow descriptions"*
- *"Add specific error messages and expected outcomes for each scenario"*

3.3 Validation and Refinement Process

AI-Generated Content Validation:

- All AI-generated scenarios were manually reviewed for technical accuracy and completeness
- Error messages were standardized to match common web application patterns
- Flow descriptions were enhanced with specific technical details (HTTP methods, status codes, API endpoints)
- Scenario IDs were systematically organized for better traceability

Manual Refinements Applied:

- Added specific preconditions and postconditions to each test scenario
- Enhanced error handling scenarios with realistic system failure conditions
- Included role-based access control specifications
- Standardized expected outcome formats for consistency

3.4 Test Case Classification

AI-Generated Test Cases:

- S1-01: Successful login (base scenario structure)
- S1-02, S1-03: Empty field validations (validation pattern)
- S2-01: View all users (CRUD operation template)
- S2-02: Add valid user (success path template)

Manually Created Test Cases:

- S1-04: Invalid email format (specific validation logic)
- S1-05, S1-06: Authentication error scenarios (domain-specific)
- S1-07: System error during login (reliability testing)
- S2-03: Add user with existing email (business rule validation)
- S2-04, S2-05: Edit and delete operations (complete CRUD coverage)
- S2-06: Missing required fields (comprehensive validation)
- S2-07: Non-admin access (security testing)
- S2-08: Delete own admin account (business logic constraint)
- S2-09: Backend failure scenarios (system resilience testing)

Quality Assurance Notes:

- All scenarios were cross-referenced with actual system requirements

- Error messages were verified against UI/UX standards
- Security considerations were manually added to ensure comprehensive coverage
- Test case dependencies and execution order were manually optimized

4 Self-Evaluation

Criteria	Self-Evaluation	Notes
At least 2 Scenario Selection	1.0 / 1.0	Selected two relevant and realistic user scenarios from The Toolshop system.
Scenario Testing	2.0 / 2.0	Clearly described scenarios and applied scenario-based thinking.
Use of AI Tools	1.0 / 1.0	Stated tool, prompts used, validation process, and usage scope.
Test Execution	0.5 / 0.5	Executed all designed test cases locally and documented results.
Bug Reporting	0.5 / 0.5	Reported bugs with full reproduction steps and severity analysis.
Scenario Testing Report	1.0 / 1.0	Report is clear and traceable